- Вступление
- Обзор и иерархия
  - Что такое коллекция
  - Иерархия интерфейсов
  - Основные методы интерфейсов
  - SequencedCollection (Java 21)
  - Collection vs Collections
- Итераторы
  - Iterable, Iterator и for-each
  - ListIterator
  - fail-fast и ConcurrentModificationException
  - fail-safe и weakly consistent
  - Как удалять элементы во время обхода
- List
  - ArrayList — устройство
  - LinkedList — устройство
  - ArrayList vs LinkedList
  - Память на один элемент
  - Vector — устарел
  - Ловушки List
- Queue и Deque
  - Queue и Deque — методы
  - ArrayDeque
  - PriorityQueue
  - Stack — устарел

- Map
  - Устройство HashMap
  - Коллизии и деревья
  - Расширение — resize
  - Ключи HashMap
  - Метод цепочек vs открытая адресация
  - LinkedHashMap и LRU-кэш
  - TreeMap и NavigableMap
  - Hashtable, WeakHashMap, IdentityHashMap, EnumMap
  - Перебор Map
- Set
  - HashSet и LinkedHashSet
  - TreeSet
  - EnumSet
- Сортировка — Comparable и Comparator
- Неизменяемые и потокобезопасные коллекции
  - Неизменяемые коллекции
  - Синхронизированные и concurrent-коллекции
- Утилиты Collections и Arrays
- Сложность операций — шпаргалка
- Вопросы к собеседованию
  - Иерархия и итераторы — вопросы
  - List — вопросы
  - Queue и Deque — вопросы
  - Map — вопросы
  - Set и сортировка — вопросы
  - Неизменяемые и потокобезопасные — вопросы

- Задачи с кодом

Вступление

Конспект по Java Collections Framework (JCF): интерфейсы, устройство основных реализаций, итераторы, сложность операций и ловушки. Ориентир — **Java 21 / 25**.

Темы идут от общего к частному:

- **Обзор и итераторы** — из каких интерфейсов состоит JCF и как устроен обход;
- **List, Queue/Deque, Map, Set** — как устроены реализации и когда какую брать;
- **Сортировка, неизменяемые и потокобезопасные коллекции** — то, на чём чаще всего ошибаются;
- **Шпаргалка по сложности** и **вопросы к собеседованию**.

Что вынесено в отдельные конспекты: `equals` / `hashCode` и их контракт — **Java Core**; `ConcurrentHashMap`, `BlockingQueue` и прочие concurrent-коллекции подробно — **Java Concurrency**; стримы над коллекциями — **Java Stream**.

Обзор и иерархия

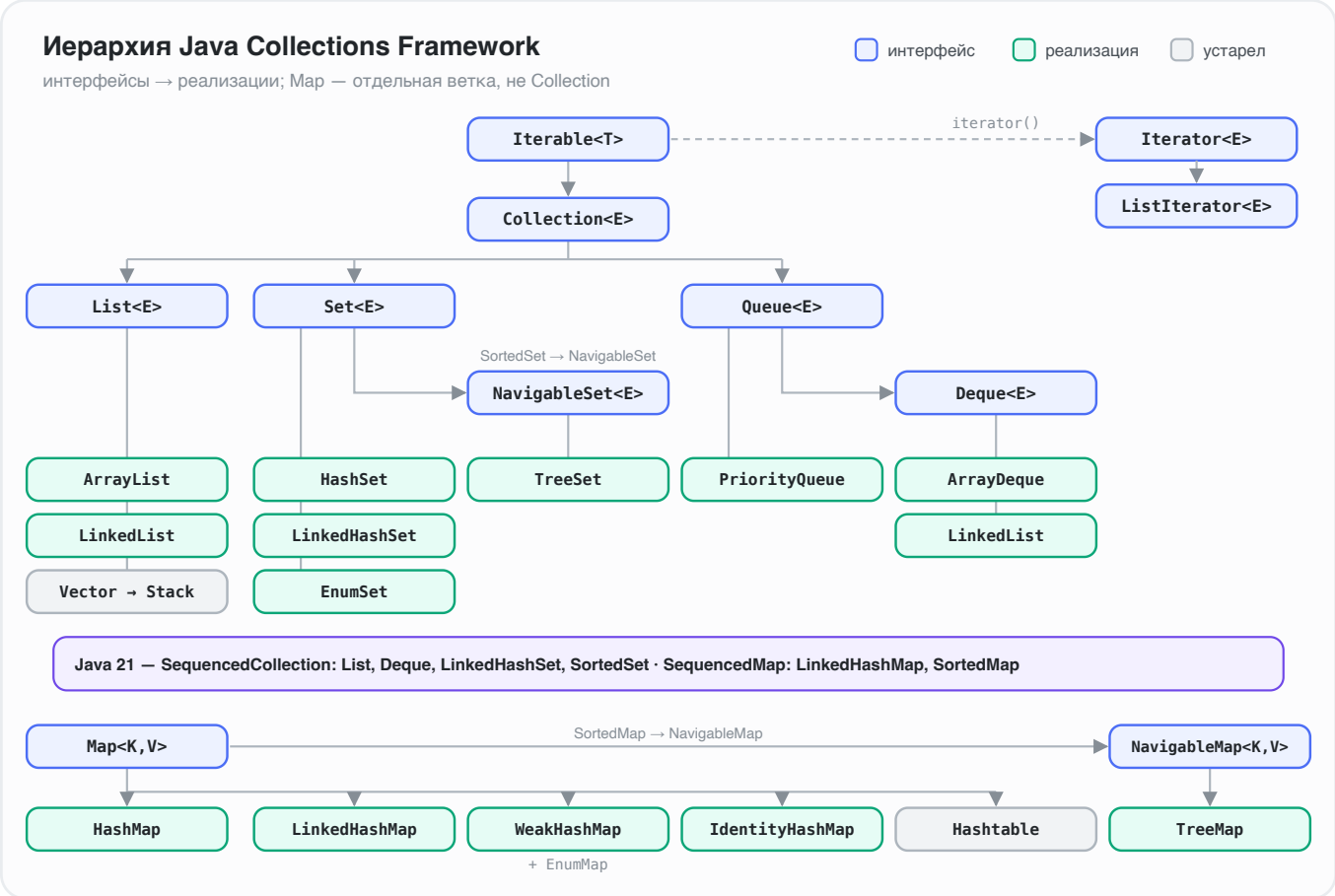
Что такое коллекция

Коллекция — объект-контейнер, который хранит группу элементов и даёт операции над ними: добавить, удалить, найти,

перебрать. **Java Collections Framework** — набор интерфейсов (`List`, `Set`, `Queue`, `Map`), их реализаций (`ArrayList`, `HashMap` ...) и утилит (`Collections`, `Arrays`) в пакете `java.util`.

Коллекции хранят только **объекты**: примитивы автоматически упаковываются (`List<Integer>`, а не `List<int>`).

Иерархия интерфейсов




```
Iterable<T>
└── Collection<E>
    ├── SequencedCollection<E>          (Java 21)
    │   ├── List<E>                     ArrayList,
LinkedList, Vector → Stack
    │   └── Deque<E>                    ArrayDeque,
LinkedList
```

```
    ├── Set<E>                                  HashSet,
LinkedHashSet, EnumSet
    │   └── SortedSet<E> → NavigableSet<E>      TreeSet
    └── Queue<E>                                PriorityQueue
        └── Deque<E>

Map<K,V>                                        HashMap,
LinkedHashMap, Hashtable, WeakHashMap, IdentityHashMap,
EnumMap
├── SequencedMap<K,V>                           (Java 21)
LinkedHashMap
└── SortedMap<K,V> → NavigableMap<K,V>  TreeMap

Iterator<E> → ListIterator<E>                   (отдельная
ветка: «курсор», а не коллекция)
```

Интерфейс	Что гарантирует	Основные реализации
`List`	порядок, доступ по индексу, дубликаты разрешены	`ArrayList`, `LinkedList`
`Set`	без дубликатов (по `equals`)	`HashSet`, `LinkedHashSet`, `TreeSet`
`Queue`	извлечение с головы: FIFO или по приоритету	`ArrayDeque`, `PriorityQueue`
`Deque`	вставка и извлечение с обоих концов: и очередь, и стек	`ArrayDeque`, `LinkedList`
`Map`	пары «ключ → значение», ключи уникальны	`HashMap`, `LinkedHashMap`, `TreeMap`

Почему `Map` — не `Collection`: `Collection` — набор отдельных элементов с методом `add(E)`, а `Map` хранит пары и работает через `put(K, V)`. Сигнатуры не совместить. Зато у `Map` есть три **представления-коллекции**: `keySet()`, `values()`, `entrySet()`.

Основные методы интерфейсов

Collection (`Collection` сама не имеет прямой реализации в JDK — только через `List`, `Set`, `Queue`):

```
size(); isEmpty(); contains(o);
add(e); remove(o); clear();
containsAll(c); addAll(c); removeAll(c); retainAll(c);
// операции над множествами
removeIf(e -> e.isBlank());
// Java 8
toArray(); toArray(String[]::new);
// Java 11
stream(); parallelStream(); iterator();
forEach(action);
```

List — добавляет работу по индексу:

```
get(i); set(i, e); add(i, e); remove(i);
indexOf(o); lastIndexOf(o);
subList(from, to);                    //
представление, а не копия
listIterator();
replaceAll(String::trim);
sort(Comparator.naturalOrder());
```

Queue — каждая операция в двух вариантах: бросить исключение или вернуть специальное значение.

	Исключение	Специальное значение
Вставить	`add(e)`	`offer(e)` → `false`
Извлечь	`remove()`	`poll()` → `null`
Посмотреть	`element()`	`peek()` → `null`

Map:

```
get(k); getOrDefault(k, def); containsKey(k);
containsValue(v);
put(k, v); putIfAbsent(k, v); remove(k); remove(k, v);
replace(k, v); replaceAll((k, v) -> v * 2);
compute(k, f); computeIfAbsent(k, k2 -> new ArrayList<>
()); computeIfPresent(k, f);
merge(k, 1, Integer::sum);                    // подсчёт
частот
keySet(); values(); entrySet(); forEach((k, v) -> ...);
Map.of(...); Map.ofEntries(Map.entry(k, v));
Map.copyOf(m);
```

Подробно про `merge`, `compute*` и фабрики `List.of` — в конспекте **Java Core** («Новые методы Map»).

SequencedCollection (Java 21)

SequencedCollection — коллекция с определённым порядком элементов и единым API для обоих концов. Раньше первый элемент `List` брали через `get(0)`, у `Deque` — через `getFirst()`, у `LinkedHashSet` — через `iterator().next()`.

```
list.getFirst();  list.getLast();          // вместо
list.get(0) и list.get(list.size() - 1)
list.addFirst(x); list.addLast(x);
list.removeFirst(); list.removeLast();
list.reversed();                            //
представление в обратном порядке, не копия
```

Интерфейс	Реализуют
`SequencedCollection`	`List`, `Deque`
`SequencedSet`	`LinkedHashSet`, `SortedSet` (`TreeSet`)
`SequencedMap`	`LinkedHashMap`, `SortedMap` (`TreeMap`): `firstEntry()`, `pollLastEntry()`, `putFirst()`, `sequencedKeySet()`

`HashSet` и `HashMap` — **не** sequenced: порядок не определён. Для неизменяемых коллекций (`List.of`) и сортированных (`TreeSet.addFirst`) методы-модификаторы бросают `UnsupportedOperationException`.

Collection vs Collections

- **`Collection`** — корневой интерфейс коллекций.
- **`Collections`** — утилитный класс со статическими методами: `sort`, `reverse`, `shuffle`, `min`, `max`, `frequency`, `binarySearch`, `unmodifiableList`, `synchronizedMap`, `emptyList` и т.д. (см. Утилиты Collections и Arrays).

Итераторы

Iterable, Iterator и for-each

Iterable — «то, что можно перебрать»: единственный абстрактный метод `iterator()`. Плюс `default`-методы `forEach(action)` и `spliterator()`.
Iterator — курсор, который идёт по элементам: `hasNext()`, `next()`, `remove()` (опционально), `forEachRemaining(action)`.

Iterable — фабрика итераторов, Iterator — сам проход. Любой `Iterable` работает в **for-each**: компилятор превращает цикл в работу с итератором.

```
for (String s : list) {                    // то, что пишем
    System.out.println(s);
}

Iterator<String> it = list.iterator();  // во что компилируется
while (it.hasNext()) {
    String s = it.next();
    System.out.println(s);
}
```

Для массивов for-each компилируется в обычный цикл по индексу.

Поведение итератора:

- `next()` без следующего элемента → `NoSuchElementException`;

- `hasNext()` только проверяет и ничего не сдвигает: 10 вызовов `hasNext()` подряд не пропускают ни одного элемента;
- `remove()` удаляет элемент, который вернул последний `next()`; вызов без `next()` или дважды подряд → `IllegalStateException`;
- у неизменяемых коллекций `remove()` → `UnsupportedOperationException`.

Spliterator («split + iterator») — итератор, который умеет делиться на части (`trySplit()`) для параллельной обработки. На нём построены `parallelStream()` — см. **Java Stream**.

Enumeration — устаревший предшественник `Iterator` из `Vector` и `Hashtable`: длинные имена (`hasMoreElements()`, `nextElement()`) и нет удаления. С Java 9 есть `enumeration.asIterator()`.

ListIterator

ListIterator — итератор только для `List`, расширяет `Iterator`:

- ходит в **обе стороны**: `hasPrevious()`, `previous()`;
- знает позицию: `nextIndex()`, `previousIndex()`; курсор стоит **между** элементами;
- меняет список при обходе: `add(e)`, `set(e)`, `remove()`;
- можно начать с позиции: `list.listIterator(list.size())` — обход с конца.

```
ListIterator<String> it = list.listIterator();
while (it.hasNext()) {
```

```
    String s = it.next();
    if (s.isBlank()) it.remove();
    else it.set(s.trim());              // заменить
текущий элемент
}
```

fail-fast и ConcurrentModificationException

fail-fast итератор — бросает `ConcurrentModificationException` (CME), если коллекцию **структурно** изменили (добавили или удалили элемент) не через этот итератор. Так у `ArrayList`, `HashMap`, `HashSet` и большинства коллекций `java.util`.

Как устроено:

- у коллекции есть счётчик `modCount`, каждое `add` / `remove` его увеличивает;
- итератор при создании запоминает `expectedModCount = modCount`;
- `next()` сравнивает их и при расхождении бросает CME.

```
List<String> list = new ArrayList<>(List.of("A", "B",
"C", "D"));
for (String s : list) {
    if (s.equals("B")) list.remove(s);   // ❌ CME на
следующем next()
}
```

⚠ CME — не гарантия

Проверка «best effort»: из нескольких потоков без синхронизации CME может не возникнуть, а данные

испортятся. Есть и однопоточная ловушка: удаление **предпоследнего** элемента в for-each не бросает CME — `hasNext()` сравнивает курсор с уменьшившимся `size` и цикл молча заканчивается, последний элемент не обработан.

`set(i, e)` — не структурное изменение, `modCount` не меняет.

fail-safe и weakly consistent

Итераторы concurrent-коллекций CME не бросают:

Вид	Как работает	Пример
Snapshot	обходит копию массива на момент создания итератора; изменений не видит; `it.remove()` не поддерживается	`CopyOnWriteArrayList`, `CopyOnWriteArraySet`
Weakly consistent	обходит живую структуру без копии; может увидеть часть изменений, сделанных после создания; каждый элемент — не больше одного раза	`ConcurrentHashMap`, `ConcurrentLinkedQueue`, `ConcurrentSkipListMap`

Термин «fail-safe» неофициальный: в документации JDK есть только fail-fast и weakly consistent. Утверждение «fail-safe итератор всегда работает с копией» верно только для copy-on-write коллекций.

Как удалять элементы во время обхода

```
list.removeIf(s -> s.isBlank());                    // ✅
лучший вариант (Java 8)

Iterator<String> it = list.iterator();              // ✅
через сам итератор
while (it.hasNext()) {
    if (it.next().isBlank()) it.remove();
}

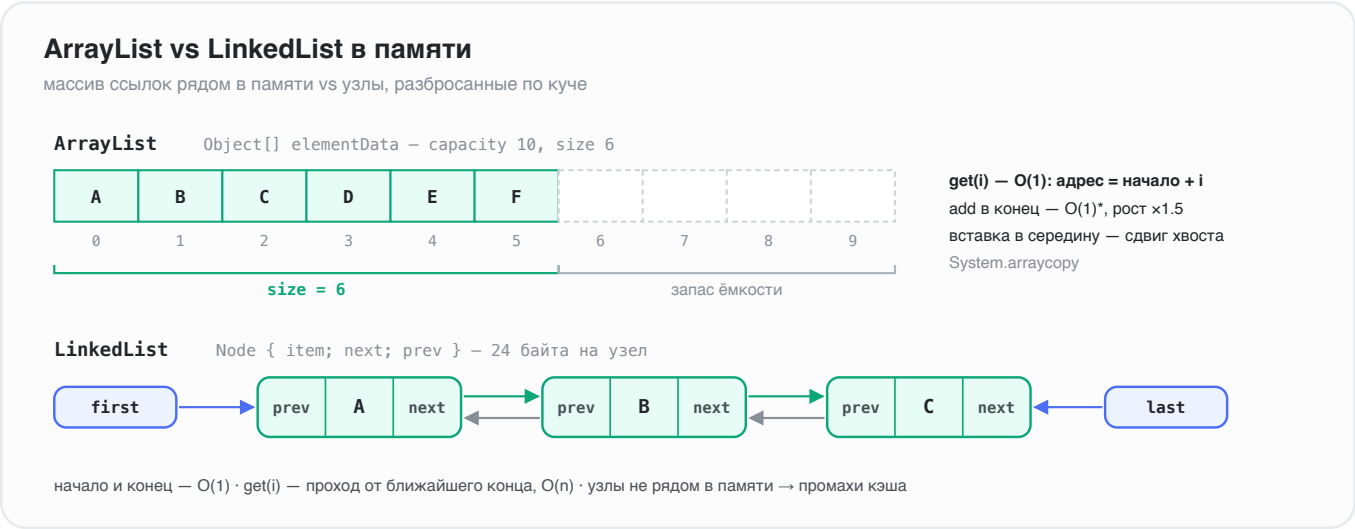
map.entrySet().removeIf(e -> e.getValue() == 0); // ✅
для Map
map.values().removeIf(v -> v == 0);

List<String> result = list.stream()                  // ✅
собрать новую коллекцию
        .filter(s -> !s.isBlank())
        .toList();
```

Ещё варианты: обходить копию (`new ArrayList<>(list)`), обходить по индексу **с конца**, использовать concurrent-коллекцию. Обход по индексу с начала с `remove(i)` пропускает элементы.

List

List — упорядоченная коллекция с доступом по индексу; дубликаты и `null` разрешены.



ArrayList — устройство

ArrayList — динамический массив: элементы лежат в `Object[] elementData`, отдельно хранится `size` — сколько элементов реально занято.

- `elementData.length` — **ёмкость (capacity)**, `size` — число элементов; ёмкость почти всегда больше. Поэтому `size` хранят отдельно.
- `new ArrayList<>()` создаёт пустой массив; ёмкость 10 выделяется при первом `add` (ленивая инициализация).
- Когда места нет — **рост в 1,5 раза**: `newCapacity = old + (old >> 1)`, элементы копируются через `Arrays.copyOf`. Поэтому `add` в конец — **амортизированно O(1)**, а в момент расширения — O(n).
- Удаление сдвигает хвост влево через `System.arraycopy`, ёмкость **не уменьшается**. Сжать вручную — `trimToSize()`, заранее выделить — `ensureCapacity(n)` или `new ArrayList<>(n)`.
- Реализует маркерный `RandomAccess` — алгоритмы `Collections` выбирают по нему доступ по индексу вместо

итератора.

```
var list = new ArrayList<Integer>(1_000);       //
знаем размер — сразу без расширений
list.subList(10, 20).clear();                   //
удалить диапазон: один сдвиг хвоста, а не 10
```

Удалять диапазон по одному элементу через `remove(i)` в цикле — O(n·k): хвост сдвигается k раз. `subList(from, to).clear()` вызывает внутренний `removeRange` и сдвигает хвост один раз.

LinkedList — устройство

LinkedList — **двусвязный** список: каждый элемент обёрнут в узел `Node { E item; Node next; Node prev; }`, список хранит ссылки `first` и `last` и `size`.

- Вставка и удаление в начале и конце — O(1).
- `get(i)` идёт от ближайшего конца — O(n), даже если это «всего» n/2 шагов.
- Вставка в середину — O(1) на саму вставку, но O(n) на поиск позиции. Выигрыш есть, только если позиция уже найдена итератором (`ListIterator.add`).
- Реализует и `List` , и `Deque` : двусвязная структура даёт O(1) на обоих концах. Обход с конца без `get(i)` — `descendingIterator()` или `reversed()` (Java 21).
- Разрешает `null` (в отличие от `ArrayDeque`).

ArrayList vs LinkedList

Операция	`ArrayList`	`LinkedList`
`get(i)` / `set(i)`	**O(1)**	O(n)
`add(e)` в конец	O(1) амортизированно	O(1)
`add(0, e)` / `removeFirst`	O(n) — сдвиг	**O(1)**
вставка / удаление в середине	O(n) — сдвиг	O(n) — поиск + O(1)
`contains` / `indexOf`	O(n)	O(n)
память на элемент	4–8 байт ссылки + запас ёмкости	узел 24–40 байт
кэш процессора	элементы рядом — быстро	узлы разбросаны по куче — промахи кэша

По умолчанию — `ArrayList`. На практике он быстрее `LinkedList` почти везде, даже при вставке в середину: `System.arraycopy` двигает непрерывный блок памяти очень быстро, а проход по узлам `LinkedList` — это промахи кэша на каждом шаге.

Нужна очередь или стек — `ArrayDeque`, а не `LinkedList`. `LinkedList` оправдан редко: частые вставки и удаления **через уже стоящий на месте `ListIterator`**.

Память на один элемент

Оценка для 64-битной JVM с **compressed oops** (по умолчанию при куче меньше 32 ГБ): заголовок объекта 12 байт, ссылка 4

байта, объекты выравниваются до 8 байт.

	`ArrayList`	`LinkedList`
Структура на элемент	ссылка в массиве: **4 байта** (+ незанятая ёмкость)	узел `Node`: 12 + 3 × 4 = 16 → **24 байта**
Без compressed oops	8 байт	16 + 3 × 8 = **40 байт**
Сам элемент `Byte`	**0 дополнительно**: `Byte.valueOf` кэширует все 256 значений `−128…127`	так же 0

Отдельный объект `Byte` занимает 16 байт (12 заголовок + 1 + выравнивание), но для `byte` он всегда берётся из кэша. Для `Integer` за пределами `−128…127` каждый элемент — ещё +16 байт.

При `add` у `ArrayList` дополнительная память нужна только при расширении — новый массив в 1,5 раза больше. У `LinkedList` каждый `add` создаёт один новый `Node`.

Vector — устарел

Vector — динамический массив из Java 1.0: все методы `synchronized`, растёт в **2 раза**. `ArrayList` (Java 1.2) появился, потому что синхронизация каждого метода стоит дорого, а в однопоточном коде не нужна. Да и в многопоточном её мало: `if (!v.contains(x)) v.add(x)` — всё равно гонка.

Нужна потокобезопасность — `CopyOnWriteArrayList`, `Collections.synchronizedList` или внешняя синхронизация

(см. Синхронизированные и concurrent-коллекции).

Ловушки List

```
List<Integer> nums = new ArrayList<>(List.of(10, 20,
30));
nums.remove(1);                          // удалит по
ИНДЕКСУ 1 → [10, 30]
nums.remove(Integer.valueOf(10));    // удалит по
ЗНАЧЕНИЮ → [30]
```

- `subList` — **представление**: изменения видны в исходном списке и наоборот. Структурное изменение исходного списка после создания `subList` → CME при обращении к `subList`.
- Цикл `for (int i = 0; i < list.size(); i++) list.get(i)` по `LinkedList` — **O(n²)**. Для любого списка безопаснее for-each.
- `Arrays.asList` и `List.of` — не `ArrayList`: см. Неизменяемые коллекции.

Способы перебрать список: for-each, `Iterator`, `ListIterator` (в обе стороны), цикл по индексу (только для `RandomAccess`), `list.forEach(...)`, `stream()`.

Queue и Deque

Queue и Deque — методы

Queue — очередь: элементы извлекаются с **головы**. Обычно **FIFO** (первым пришёл — первым ушёл), но `PriorityQueue`

отдаёт по приоритету.
Deque (double-ended queue) — **расширяет Queue**: вставка и извлечение с **обоих концов**. Поэтому `Deque` — и очередь (FIFO), и стек (**LIFO**).

	Голова (first)	**Хвост (last)**
Вставить	`addFirst` / `offerFirst` / `push`	`addLast` / `offerLast` / `add` / `offer`
Извлечь	`removeFirst` / `pollFirst` / `pop` / `poll`	`removeLast` / `pollLast`
Посмотреть	`getFirst` / `peekFirst` / `peek`	`getLast` / `peekLast`

```
Deque<Integer> stack = new ArrayDeque<>();   // LIFO
stack.push(1); stack.push(2);
stack.pop();                                 // 2

Queue<Integer> queue = new ArrayDeque<>();   // FIFO
queue.offer(1); queue.offer(2);
queue.poll();                                // 1
```

`Queue` и `Deque` обычно **не** переопределяют `equals` / `hashCode` — сравнение по ссылке (кроме `LinkedList`, который наследует их от `List`).

Блокирующие очереди для producer-consumer (`ArrayBlockingQueue`, `LinkedBlockingQueue`) — в конспекте **Java Concurrency**.

ArrayDeque

ArrayDeque — `Deque` на **кольцевом массиве** с индексами `head` и `tail`: операции на обоих концах — O(1) амортизированно, без узлов и без сдвигов.

- Лучший выбор и для стека, и для очереди: быстрее `Stack` (нет синхронизации) и `LinkedList` (нет узлов, данные рядом в памяти).
- **`null` запрещён** — `poll()` возвращает `null` как признак пустой очереди, иначе их не различить.
- Не потокобезопасен; доступа по индексу нет.

PriorityQueue

PriorityQueue — очередь с приоритетом на **двоичной куче** (binary heap) в массиве: голова — всегда **минимальный** элемент по `Comparable` или переданному `Comparator`.

Операция	Сложность
`offer` / `poll`	O(log n)
`peek`	O(1)
`remove(o)` / `contains`	O(n)

```
var tasks = new PriorityQueue<Task>
(Comparator.comparingInt(Task::priority));
var maxHeap = new PriorityQueue<Integer>
(Comparator.reverseOrder());   // максимум в голове

// топ-3 самых больших: держим min-heap размера 3
var top = new PriorityQueue<Integer>();
for (int x : nums) {
    top.offer(x);
```

```
    if (top.size() > 3) top.poll();
}
```

- **Итератор и `toString()` не отсортированы** — по порядку отдаёт только `poll()`.
- `null` запрещён; не потокобезопасна (есть `PriorityBlockingQueue`).
- Применение: планировщики задач, алгоритм Дейкстры, слияние k отсортированных списков, топ-K.

Stack — устарел

Stack (Java 1.0) наследует `Vector`:

- все методы синхронизированы (`push` вызывает синхронизированный `addElement`) — лишняя цена;
- наследование ломает абстракцию стека: доступны `get(i)`, `add(i, e)`, `remove(i)` в середине.

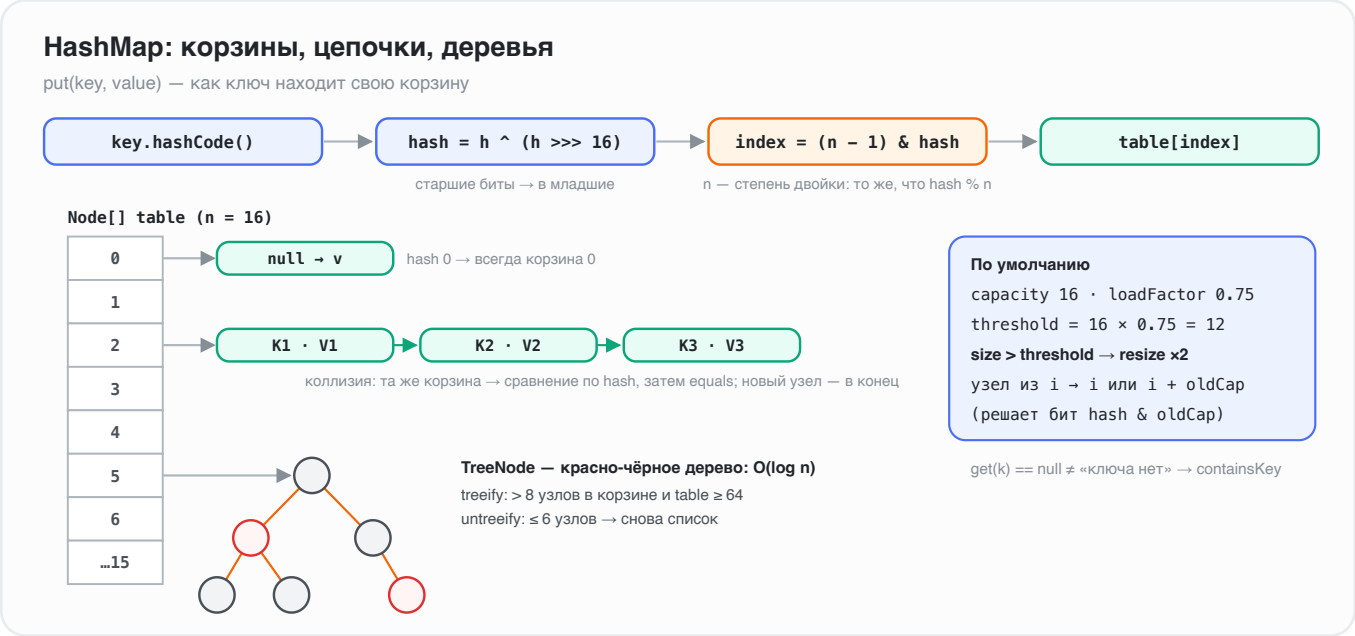
Замена — `Deque`: `Deque<T> stack = new ArrayDeque<>(); push / pop / peek`. Для многопоточного стека — `ConcurrentLinkedDeque`.

Map

Map — отображение «ключ → значение»: ключи уникальны (по `equals` или по `compareTo` у `TreeMap`), значения могут повторяться.

	`HashMap`	`LinkedHashMap`	`TreeMap`
Порядок обхода	не определён	вставки (или доступа)	отсортирован по ключу
`get` / `put`	O(1) в среднем	O(1) в среднем	O(log n)
`null`-ключ	один	один	нет (при natural ordering — NPE)
Внутри	массив корзин	`HashMap` + двусвязный список	красно-чёрное дерево

Устройство HashMap



HashMap — хэш-таблица: массив **корзин** (`Node<K,V>[] table`), в каждой — цепочка узлов `Node { int hash; K key; V value; Node next; }`.

`put(key, value)`:

1. `hash = key.hashCode() ^ (h >>> 16)` — старшие биты подмешиваются к младшим, иначе для индекса работали бы только младшие биты хэша.
2. Индекс корзины: `index = (n - 1) & hash`, где `n` — размер таблицы, **всегда степень двойки**. Это то же, что `hash % n`, но быстрее и без отрицательных результатов.
3. Корзина пуста — кладём новый `Node`.
4. Иначе идём по цепочке: нашли ключ (`hash` совпал **и** `key == k || key.equals(k)`) — заменяем значение и возвращаем старое; не нашли — добавляем узел **в конец** цепочки.
5. `++size > threshold` — расширяем таблицу.

`get(key)` — те же шаги 1–2, затем сравнение с узлами корзины: сначала по `hash`, потом по `equals`. Если ключ лежит первым в корзине — это вычисление хэша, индекса и одно сравнение.

- `null`-ключ разрешён один: его хэш 0, он всегда в корзине 0. Значений `null` — сколько угодно. Поэтому `get(k) == null` не значит «ключа нет» — проверяй `containsKey`.
- **Таблица создаётся лениво** — при первом `put`. По умолчанию ёмкость **16**, `loadFactor` **0.75**, порог расширения `threshold = 16 × 0.75 = 12`.
- `new HashMap<>(n)` округляет `n` вверх до степени двойки. Чтобы положить `n` элементов без расширений, нужна ёмкость `n / 0.75 + 1` — с Java 19 это считает `HashMap.newHashMap(n)`.
- Новый ключ создаёт **один** объект `Node` (до Java 8 он назывался `Entry`).

- Порядок обхода не определён и может меняться после расширения.

Коллизии и деревья

Коллизия — разные ключи попали в одну корзину: одинаковый `hashCode` или разный `hashCode`, но одинаковый индекс `(n - 1) & hash`. Ключи с одинаковым `hashCode`, но `equals() == false`, просто лежат в одной цепочке — `get` найдёт нужный по `equals`.

С **Java 8** длинная цепочка превращается в **красно-чёрное дерево** (`TreeNode`):

- **treeify** — когда в корзине больше `TREEIFY_THRESHOLD = 8` узлов **и** таблица не меньше `MIN_TREEIFY_CAPACITY = 64`. Если таблица меньше 64 — вместо дерева она расширяется;
- **untreeify** — дерево обратно в список при `≤ 6` узлах после расширения или удаления.

Поэтому поиск в «плохой» корзине — **O(log n)** вместо O(n), что защищает и от атак на хэш-коллизии.

Для дерева ключи нужно сравнивать, но `Comparable` от ключей не требуется. Порядок в дереве определяется так:

1. по значению `hash`;
2. при равенстве — через `compareTo`, если ключи `Comparable` одного класса;
3. иначе `tieBreakOrder`: по имени класса, затем по `System.identityHashCode`.

Сложность HashMap: в среднем O(1), но **не гарантированно**: при плохом `hashCode` (например, константа) все ключи в одной

корзине — O(log n) для `Comparable` ключей и O(n) по сути для остальных; до Java 8 — O(n). Мапа при этом **работает корректно**, просто медленно.

Расширение — resize

Когда `size > capacity × loadFactor`, таблица **удваивается**, и все узлы перераспределяются. Благодаря степени двойки пересчитывать индекс полностью не нужно: узел из корзины `i` попадает либо в `i`, либо в `i + oldCapacity` — решает один бит `hash & oldCapacity`. Цепочка делится на две («lo» и «hi») без пересчёта хэшей.

`HashMap(int initialCapacity, float loadFactor)`:

- `initialCapacity` — начальное число корзин (округлится до степени двойки);
- `loadFactor` — доля заполнения, при которой таблица расширяется. Меньше — меньше коллизий, но больше памяти; больше — наоборот. 0.75 — компромисс по умолчанию.

Расширение — O(n), поэтому при известном размере задавайте ёмкость заранее.

> ⚠ **HashMap не потокобезопасен**
>
> Параллельные `put` теряют записи и могут испортить структуру. До Java 8 вставка шла в начало цепочки, и одновременный resize мог зациклить список — `get` уходил в

бесконечный цикл. Для многопоточности — `ConcurrentHashMap`.

Ключи HashMap

Ключ должен иметь корректные `equals` и `hashCode` по одним и тем же полям и быть **неизменяемым** — контракт подробно в конспекте **Java Core**.

Как потерять элемент: изменить поле ключа, которое участвует в `hashCode`, после вставки. Узел лежит в корзине по **старому** хэшу, а `get` ищет по новому — в другой корзине. Даже если корзина совпала, сравнение по сохранённому `hash` не сработает.

```
var key = new ArrayList<>(List.of(1));
Map<List<Integer>, String> map = new HashMap<>();
map.put(key, "v");
key.add(2);                          // hashCode ключа
изменился
map.get(key);                        // null
map.containsKey(List.of(1));         // false — equals уже
не совпадает
```

Почему `byte[]` — плохой ключ: у массивов `equals` и `hashCode` унаследованы от `Object` — сравнение по ссылке. Другой массив с тем же содержимым ключ не найдёт.

```
byte[] a = {1, 2, 3};
byte[] b = {1, 2, 3};
map.put(a, "hello");
```

```
map.get(a);                    // hello
map.get(b);                    // null
```

Решение — обернуть: `ByteBuffer.wrap(bytes)`, `List.of(...)`, `record` с `Arrays.equals` / `Arrays.hashCode` или строка (Base64).

`String` — идеальный ключ: неизменяемый и кэширует `hashCode`. Различных значений `hashCode` всего 2^{32} (диапазон `int`), поэтому коллизии неизбежны.

Метод цепочек vs открытая адресация

Две классические реализации хэш-таблицы (Кнут, Кормен):

	Метод цепочек	**Открытая адресация**
Где элементы	в цепочке (списке / дереве) у корзины	прямо в массиве
Коллизия	добавить в цепочку	искать другую свободную ячейку: линейное, квадратичное пробирование, двойное хэширование
Заполнение	может быть > 1, деградирует плавно	≤ 1, при росте заполнения резко падает скорость
Удаление	простое	сложное — нужны метки-«надгробия»
Память	объект-узел на каждый элемент	без узлов, данные рядом — дружелюбно к кэшу
В JDK	`HashMap`, `Hashtable`,	`IdentityHashMap` (линейное

	Метод цепочек	Открытая адресация
	`ConcurrentHashMap`	пробирование), `ThreadLocalMap`

`HashMap` использует цепочки: проще удаление, устойчивость к высокому заполнению и возможность превращать цепочки в деревья.

LinkedHashMap и LRU-кэш

LinkedHashMap — `HashMap` + **двусвязный список** через все записи. От `HashMap` — хэш-таблица и O(1), от `LinkedList` — предсказуемый порядок обхода.

- По умолчанию **insertion-order**: порядок вставки. Повторный `put` существующего ключа порядок **не меняет**.
- `new LinkedHashMap<>(16, 0.75f, true)` — **access-order**: `get` и `put` переносят запись в конец списка; в голове — давно не использованные.
- `removeEldestEntry(eldest)` — вызывается после каждого `put`; вернёт `true` — самая старая запись удаляется.

LRU-кэш (least recently used, «вытеснять давно не использованное»):

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int maxSize;

    LruCache(int maxSize) {
        super(16, 0.75f, true);                  //
access-order
        this.maxSize = maxSize;
```

```
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V>
eldest) {
        return size() > maxSize;                // 
переполнение → удалить голову
    }
}
```

Не потокобезопасен: даже `get` в access-order меняет структуру. В продакшене кэш берут готовый — Caffeine.

TreeMap и NavigableMap

TreeMap — `NavigableMap` на **красно-чёрном дереве** (самобалансирующемся двоичном дереве поиска): ключи всегда отсортированы по `Comparable` или переданному `Comparator`. Все основные операции — **O(log n)**.

«Сортированность» `SortedMap` — не только `toString()` по порядку: отсортированный обход, `firstKey()` / `lastKey()` и представления-диапазоны. `NavigableMap` добавляет поиск ближайших ключей:

```
TreeMap<Integer, String> rates = new TreeMap<>
(Map.of(0, "A", 1000, "B", 5000, "C"));

rates.floorKey(2500);        // 1000 — наибольший ключ
≤ 2500 (тариф по сумме)
rates.ceilingKey(2500);      // 5000 — наименьший ≥
rates.lowerKey(1000);        // 0    — строго <
rates.higherKey(1000);       // 5000 — строго >
```

```
rates.headMap(1000);          // {0=A}          — ключи
< 1000 (представление)
rates.tailMap(1000, true);    // {1000=B, 5000=C}
rates.subMap(0, 5000);        // {0=A, 1000=B}
rates.firstEntry(); rates.pollLastEntry();
rates.descendingMap();
```

- Ключи сравниваются через `compareTo` / `compare`, **не через `equals`**. Если компаратор считает ключи равными — это один ключ.
- `null`-ключ при natural ordering → `NullPointerException`.
- Добавление по возрастанию не ломает дерево: оно само балансируется, высота остаётся O(log n).
- Потокобезопасная сортированная мапа — `ConcurrentSkipListMap`.

Hashtable, WeakHashMap, IdentityHashMap, EnumMap

Hashtable (Java 1.0) — устарел. Отличия от `HashMap`: все методы `synchronized`, `null` запрещён и в ключах, и в значениях, обход через `Enumeration` (не fail-fast) или итератор. `HashMap` появился, потому что синхронизация на каждый вызов дорогая и не спасает составные операции. Замена в многопоточном коде — `ConcurrentHashMap`.

WeakHashMap — ключи хранятся через `WeakReference`. Когда на ключ не остаётся сильных ссылок, GC его собирает, и запись удаляется из мапы (через `ReferenceQueue` при следующем обращении к мапе).

Ссылка	Когда GC собирает объект
сильная (обычная)	никогда, пока ссылка жива
`SoftReference`	при нехватке памяти
`WeakReference`	при ближайшей сборке, если нет сильных и мягких ссылок
`PhantomReference`	`get()` всегда `null`; нужна только для уведомления, что объект собран (`Cleaner`)

- Применение `WeakHashMap`: прикрепить данные к чужим объектам, не мешая их сборке (метаданные, кэш по ключу-объекту).
- Ловушка: ключи-строковые литералы и закэшированные `Integer` живут вечно — записи не удалятся.
- **SoftHashMap** в JDK нет — memory-sensitive кэш лучше брать готовый (Caffeine с `softValues()`, Guava). **PhantomHashMap** бессмысленна: по фантомной ссылке объект не достать.

IdentityHashMap — сравнивает ключи по **ссылке** (`==`) и `System.identityHashCode`, а не `equals` / `hashCode`. Внутри — открытая адресация с линейным пробированием: ключи и значения лежат в одном массиве.

```
String s1 = new String("Hello");
String s2 = new String("Hello");
new HashMap<>(Map.of(s1, 1)).get(s2);       // 1 — равны по equals
var im = new IdentityHashMap<String, Integer>();
im.put(s1, 1);
```

```
im.get(s2);                                    // null — разные объекты
```

Применение: обход графов объектов, сериализация, глубокое копирование — запомнить «этот объект уже обработан», даже если `equals` говорит, что объекты равны. Требование неизменяемости ключа на неё не распространяется.

EnumMap — ключи одного enum, внутри массив по `ordinal()`: компактно, быстрее `HashMap`, обход в порядке объявления констант.

Перебор Map

```
for (Map.Entry<String, Integer> e : map.entrySet()) {
// ключ и значение — лучший вариант
    use(e.getKey(), e.getValue());
}
map.forEach((k, v) -> use(k, v));
// Java 8
for (String k : map.keySet()) { }
// только ключи
for (Integer v : map.values()) { }
// только значения
```

Перебор `keySet()` с `map.get(k)` внутри — лишний поиск на каждый ключ. `keySet()`, `values()`, `entrySet()` — **представления**: удаление через них удаляет из мапы, добавление (`add`) не поддерживается.

Set

Set — коллекция без дубликатов. Внутри почти каждый `Set` — это `Map`, где элементы — ключи.

	`HashSet`	`LinkedHashSet`	`TreeSet`	`EnumSe`
Внутри	`HashMap`	`LinkedHashMap`	`TreeMap`	битовы вектор
Порядок	не определён	вставки	отсортирован	объявл enum
`add` / `contains`	O(1)	O(1)	O(log n)	O(1)
Равенство элементов	`equals` / `hashCode`	`equals` / `hashCode`	`compareTo` / `compare`	`ordina`
`null`	можно	можно	нет	нет

HashSet и LinkedHashSet

HashSet — обёртка над `HashMap`: элемент — ключ, значение — общий объект-заглушка `PRESENT`. `add(e)` возвращает `false`, если элемент уже есть. Сложность и требования к элементам — как у ключей `HashMap`.

LinkedHashSet — то же на `LinkedHashMap`: обход в порядке вставки, повторное добавление порядок не меняет. Нужен, когда важно и убрать дубликаты, и сохранить порядок.

```
new HashSet<>(List.of(30, 10, 20));          // порядок любой
new LinkedHashSet<>(List.of(30, 10, 20));    // [30, 10, 20]
new TreeSet<>(List.of(30, 10, 20));          // [10, 20, 30]
```

TreeSet

TreeSet — `NavigableSet` на `TreeMap`: элементы отсортированы, операции O(log n), есть `first()`, `last()`, `floor(x)`, `ceiling(x)`, `headSet`, `tailSet`, `subSet`, `descendingSet()`.

- Дубликаты определяются через `compareTo` / `compare`, а **не** `equals`. Компаратор «только по возрасту» склеит разных людей одного возраста в один элемент.
- Порядок добавления не важен: дерево балансируется само.

EnumSet

EnumSet — `Set` для значений одного enum на **битовом векторе**: каждой константе соответствует бит по `ordinal()`. До 64 констант — один `long` (`RegularEnumSet`), больше — массив `long` (`JumboEnumSet`).

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
// биты: 0000011
EnumSet<Day> workdays = EnumSet.complementOf(weekend);
EnumSet<Day> mid = EnumSet.range(Day.TUE, Day.THU);
EnumSet.allOf(Day.class); EnumSet.noneOf(Day.class);
```

Чем лучше `HashSet` / `TreeSet`: в разы компактнее, все операции — битовые, O(1); `containsAll`, `retainAll` — одна битовая операция над `long`; обход — в порядке объявления.

Сортировка — Comparable и Comparator

Comparable — объект сам задаёт свой **естественный порядок**: `int compareTo(T other)`. Реализуют `String`, обёртки, `LocalDate`, `BigDecimal`, enum.
Comparator — отдельный объект «как сравнивать»: `int compare(T a, T b)`. Позволяет задать сколько угодно порядков и сортировать классы, которые нельзя изменить.

Результат: `< 0` — первый меньше, `0` — равны, `> 0` — первый больше.

```
record User(String name, String city, int age)
implements Comparable<User> {
    public int compareTo(User o) { return
Integer.compare(age, o.age); }   // не age - o.age!
}

users.sort(Comparator.comparing(User::name));
// по имени
users.sort(Comparator.comparingInt(User::age).reversed(
));                // по возрасту, убывание
users.sort(Comparator.comparing(User::city,
Comparator.nullsLast(Comparator.naturalOrder()))
                    .thenComparing(User::name));
// по нескольким полям
```

- **Не вычитай**: `a - b` переполняется (`Integer.MIN_VALUE - 1` > 0) — только `Integer.compare(a, b)`.

- Контракт: `sgn(compare(a, b)) == -sgn(compare(b, a))`, транзитивность. Нарушение → `IllegalArgumentException: Comparison method violates its general contract` из сортировки.
- **Согласованность с `equals`** желательна: `compare(a, b) == 0` ⇔ `a.equals(b)`. Иначе `TreeSet` и `HashSet` по-разному понимают дубликаты. Пример: `new BigDecimal("2.0")` и `"2.00"` — в `HashSet` два элемента, в `TreeSet` один.

Как сортирует JDK:

- `List.sort`, `Collections.sort`, `Arrays.sort(Object[])` — **TimSort**: устойчивая (stable — равные элементы сохраняют порядок), O(n log n), быстрая на частично отсортированных данных;
- `Arrays.sort(int[])` и других примитивов — **Dual-Pivot Quicksort** (устойчивость для примитивов не важна).

Неизменяемые и потокобезопасные коллекции

Неизменяемые коллекции

Способ	Что это	Измени
`List.of` / `Set.of` / `Map.of` (Java 9)	новая неизменяемая коллекция	`Unsupp`

Способ	Что это	Измени
`List.copyOf(c)` (Java 10)	неизменяемая **копия**	UOE
`stream.toList()` (Java 16)	неизменяемый список	UOE
`Collections.unmodifiableList(l)`	**представление** только для чтения	UOE
`Arrays.asList(arr)`	список **фиксированного размера** поверх массива	`set` — UOE
`Collections.emptyList()` / `singletonList(x)`	неизменяемые пустой / из одного элемента	UOE

```
List<String> src = new ArrayList<>(List.of("a"));
List<String> view = Collections.unmodifiableList(src);
List<String> copy = List.copyOf(src);
src.add("b");
view;   // [a, b] — представление видит изменения
copy;   // [a]    — копия нет

Arrays.asList(1, 2).add(3);              //
UnsupportedOperationException
Arrays.asList(new int[]{1, 2}).size();   // 1 —
List<int[]>, а не List<Integer>!
Set.of("a", "a");                        //
IllegalArgumentException: duplicate element
```

- Неизменяемость **поверхностная**: объекты внутри можно менять.
- Порядок обхода `Set.of` и `Map.of` **специально рандомизирован** между запусками JVM — не полагайтесь на него.
- `Collections.unmodifiable*` — защита API от изменений вызывающим кодом, но не неизменяемость: владелец исходной коллекции может её поменять.

Синхронизированные и concurrent-коллекции

`Collections.synchronizedList / Map / Set` — обёртка, где каждый метод синхронизирован на одном мьютексе. Минусы:

- **составные операции не атомарны**: `if (!list.contains(x)) list.add(x)` — гонка;
- **перебор** нужно синхронизировать вручную, иначе CME:

```
List<String> list = Collections.synchronizedList(new ArrayList<>());
synchronized (list) {                    // обязательно на время обхода
    for (String s : list) { ... }
}
```

Concurrent-коллекции (`java.util.concurrent`, Java 5+) — спроектированы для конкурентного доступа:

Коллекция	Когда
`ConcurrentHashMap`	общая мапа, атомарные `compute`, `merge`, `putIfAbsent`
`CopyOnWriteArrayList`	чтений много, записей мало: слушатели, конфиги
`ConcurrentLinkedQueue` / `Deque`	неблокирующая очередь
`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`)	producer-consumer
`ConcurrentSkipListMap` / `Set`	сортированная потокобезопасная мапа / множество

Как они устроены — в конспекте **Java Concurrency**.

Утилиты Collections и Arrays

```
Collections.sort(list); Collections.reverse(list);
Collections.shuffle(list);
Collections.min(c); Collections.max(c, comparator);
Collections.frequency(c, x);                // сколько раз встречается
Collections.binarySearch(sortedList, key);  // только по отсортированному списку
Collections.swap(list, i, j); Collections.fill(list, x); Collections.nCopies(3, "a");
Collections.addAll(c, "a", "b");
Collections.disjoint(c1, c2);               // нет
```

```
общих элементов
Collections.unmodifiableList(l);
Collections.synchronizedMap(m);
Collections.emptyList();

Arrays.asList(...); Arrays.sort(arr);
Arrays.binarySearch(arr, key);
Arrays.fill(arr, 0); Arrays.copyOf(arr, n);
Arrays.copyOfRange(arr, from, to);
Arrays.equals(a, b); Arrays.hashCode(a);
Arrays.toString(a); Arrays.deepToString(matrix);
Arrays.stream(arr);
```

Преобразования одной строкой:

```
Object[] arr = collection.toArray();
String[] arr2 = list.toArray(String[]::new);       //
Java 11; раньше toArray(new String[0])
List<T> list = new ArrayList<>(set);               //
Set → List
Set<T> set = new HashSet<>(list);                  //
List → Set (убрать дубликаты)
Set<K> keys = new HashSet<>(map.keySet());         //
копия ключей, а не представление
List<T> middle = list.subList(3, list.size() - 3); //
без первых и последних трёх (представление)
```

Сложность операций — шпаргалка

Коллекция	Добавить	Удалить	Найти / `contains`
`ArrayList`	O(1)* в конец, O(n) в середину	O(n)	O(n)
`LinkedList`	O(1) на концах	O(1) на концах, O(n) в середине	O(n)
`ArrayDeque`	O(1)* на концах	O(1) на концах	O(n)
`PriorityQueue`	O(log n)	O(log n) голова, O(n) произвольный	O(n)
`HashSet` / `HashMap`	O(1)**	O(1)**	O(1)**
`LinkedHashSet` / `LinkedHashMap`	O(1)**	O(1)**	O(1)**
`TreeSet` / `TreeMap`	O(log n)	O(log n)	O(log n)
`EnumSet` / `EnumMap`	O(1)	O(1)	O(1)
`CopyOnWriteArrayList`	O(n) — копия массива	O(n)	O(n)

* амортизированно — иногда O(n) на расширение массива. ** в среднем; худший случай — O(log n) для корзины-дерева.

Вопросы к собеседованию

Иерархия и итераторы — вопросы

Назовите основные интерфейсы JCF и их реализации.

`Collection` → `List` (`ArrayList`, `LinkedList`), `Set` (`HashSet`, `LinkedHashSet`, `TreeSet`), `Queue` / `Deque` (`ArrayDeque`, `PriorityQueue`). Отдельно `Map` (`HashMap`, `LinkedHashMap`, `TreeMap`). Устаревшие: `Vector`, `Stack`, `Hashtable`.

Почему Map — не Collection?

`Collection` — набор отдельных элементов с `add(E)`, `Map` — набор пар с `put(K, V)`; сигнатуры не совместимы. Зато `Map` даёт представления-коллекции `keySet()`, `values()`, `entrySet()`.

Collection vs Collections?

`Collection` — корневой интерфейс коллекций. `Collections` — утилитный класс со статическими методами (`sort`, `unmodifiableList`, `synchronizedMap` ...).

Как связаны Iterable, Iterator и for-each?

`Iterable.iterator()` создаёт `Iterator`, который выполняет обход (`hasNext`, `next`, `remove`). for-each работает с любым `Iterable` — компилятор превращает его в цикл по итератору.

Iterator vs ListIterator?

`ListIterator` — только для `List`: ходит в обе стороны (`previous`), знает индексы, умеет `add` и `set`. `Iterator` — только вперёд и `remove`.

Iterator vs Enumeration?

`Enumeration` — устаревший (Java 1.0, `Vector` / `Hashtable`): нет `remove`, длинные имена методов. `Iterator` есть у всех современных коллекций.

Что будет при next() без hasNext()?

Если следующий элемент есть — вернётся он; если нет — `NoSuchElementException`.

Сколько элементов пропустит next() после 10 вызовов hasNext()?

Ни одного: `hasNext()` только проверяет и курсор не двигает.

Что делает iterator.remove() и когда он бросает исключение?

Удаляет элемент, возвращённый последним `next()`. Без предшествующего `next()` или повторно — `IllegalStateException`; у неизменяемой коллекции — `UnsupportedOperationException`.

Что будет с итератором, если вызвать collection.remove()?

При следующем `next()` — `ConcurrentModificationException`: `modCount` коллекции разошёлся с `expectedModCount` итератора.

Что такое fail-fast и как он устроен?

Итератор бросает CME при структурном изменении коллекции не через него. Сравнивает `modCount` коллекции с запомненным

`expectedModCount`. Работает по принципу «best effort» — не гарантия.

fail-fast vs fail-safe? Примеры fail-safe.

fail-fast бросает CME. «fail-safe» не бросает: `CopyOnWriteArrayList` обходит снимок массива, итераторы `ConcurrentHashMap` и `ConcurrentLinkedQueue` — weakly consistent: обходят живую структуру без копии и могут увидеть часть изменений.

Как избежать ConcurrentModificationException?

Удалять через `iterator.remove()` или `removeIf`, собирать новую коллекцию стримом, обходить копию, использовать concurrent-коллекции. В многопоточном коде — синхронизация или `ConcurrentHashMap` / `CopyOnWriteArrayList`.

Какие коллекции реализуют FIFO и LIFO?

FIFO — `Queue` (`ArrayDeque`, `LinkedList`). LIFO — `Deque` как стек (`ArrayDeque.push/pop`); устаревший вариант — `Stack`.

List — вопросы

ArrayList vs LinkedList — когда что?

`ArrayList` — массив: O(1) по индексу, вставка в середину O(n) через быстрый `arraycopy`, мало памяти. `LinkedList` — двусвязный список: O(1) на концах, O(n) по индексу, узел на каждый элемент. По умолчанию — `ArrayList`; для очереди и стека — `ArrayDeque`.

Какое худшее время contains() в ArrayList и LinkedList?

O(n) у обоих — линейный поиск по `equals`.

Какое худшее время add() в ArrayList и LinkedList?

`ArrayList` — O(n): при нехватке ёмкости копируется весь массив (в среднем O(1)); в середину — O(n) на сдвиг. `LinkedList` — O(1) в конец, O(n) по индексу из-за поиска позиции.

Где медленнее list.add(list.size() / 2, x)?

По сложности оба O(n). На практике обычно медленнее `LinkedList`: проход n/2 узлов с промахами кэша дороже, чем `System.arraycopy` половины массива.

Как ArrayList удаляет элементы и меняется ли его размер?

Хвост сдвигается влево `System.arraycopy`, `size` уменьшается, а ёмкость массива — нет. Сжать — `trimToSize()`.

Как эффективно удалить несколько подряд идущих элементов из ArrayList?

`list.subList(from, to).clear()` — хвост сдвигается один раз, а не на каждый `remove`.

Сколько дополнительной памяти нужно при ArrayList.add()?

Если ёмкости хватает — нисколько. Иначе — новый массив в 1,5 раза больше старого (на время копирования живут оба).

Сколько памяти выделяется при LinkedList.add()?

Один объект `Node` — 24 байта с compressed oops (40 без них).

Сколько памяти занимает byte в LinkedList и в ArrayList?

`LinkedList` — узел 24 байта (40 без compressed oops), `ArrayList` — ссылка 4 байта (8). Сам `Byte` дополнительно не создаётся: все 256 значений берутся из кэша `Byte.valueOf`.

Зачем ArrayList хранит size отдельно от elementData.length?

`elementData.length` — ёмкость, она обычно больше числа элементов: массив выделяется с запасом. `size` — сколько элементов реально хранится.

LinkedList — односвязный, двусвязный или четырёхсвязный?

Двусвязный: у каждого узла ссылки `next` и `prev`, у списка — `first` и `last`.

Почему LinkedList реализует и List, и Deque?

Двусвязная структура даёт O(1) на обоих концах — это и есть `Deque`. Плюс порядок и индексы — это `List`.

Как перебрать LinkedList с конца без get(i)?

`descendingIterator()`, `listIterator(list.size())` с `previous()` или `reversed()` (Java 21).

ArrayList vs Vector? Зачем добавили ArrayList?

`Vector` синхронизирует каждый метод и растёт в 2 раза, `ArrayList` — без синхронизации и растёт в 1,5 раза. `ArrayList` добавили, потому что синхронизация дорогая и не делает составные операции атомарными. `Vector` устарел.

Какие способы перебрать список?

for-each, `Iterator`, `ListIterator`, цикл по индексу (только для `ArrayList` — для `LinkedList` O(n²)), `forEach(lambda)`, `stream()`.

Queue и Deque — вопросы

Queue vs Deque? Кто кого расширяет?

`Deque` расширяет `Queue`. `Queue` — извлечение с головы (обычно FIFO), `Deque` — вставка и извлечение с обоих концов, поэтому и очередь, и стек.

offer vs add, poll vs remove, peek vs element?

`add`, `remove`, `element` бросают исключение (переполнение, пустая очередь); `offer`, `poll`, `peek` возвращают `false` / `null`.

Что позволяет PriorityQueue?

Извлекать элементы по приоритету: голова — минимальный по `Comparable` или `Comparator`. Двоичная куча: `offer` / `poll` O(log n), `peek` O(1). Итератор не отсортирован, `null` запрещён.

Чем заменить Stack и почему?

`ArrayDeque` (`push` / `pop` / `peek`). `Stack` наследует `Vector`: лишняя синхронизация всех методов и доступ к середине по индексу, что ломает абстракцию стека.

Почему ArrayDeque запрещает null?

`poll()` и `peek()` возвращают `null` как признак пустой очереди — с `null`-элементами их не различить.

Map — вопросы

Как устроен HashMap?

Массив корзин `Node[]`. Хэш ключа (`h ^ h >>> 16`) → индекс `(n - 1) & hash` → поиск по цепочке через `hash` и `equals`. Нет ключа — новый узел в конец цепочки. Длинные цепочки (> 8 при таблице ≥ 64) становятся красно-чёрными деревьями. При `size > capacity × 0.75` таблица удваивается.

Какая роль у equals() и hashCode() в HashMap?

`hashCode` выбирает корзину, `equals` находит ключ среди узлов корзины.

Что будет с двумя ключами с одинаковым hashCode, но equals() == false?

Оба лягут в одну корзину (в цепочку или дерево) и будут разными ключами.

Начальное количество корзин в HashMap?

16 по умолчанию; массив создаётся лениво при первом `put`. В конструкторе можно задать своё — округлится вверх до степени двойки.

Сложность операций HashMap и гарантирована ли она?

В среднем O(1), но не гарантированно: при коллизиях корзина-дерево даёт O(log n), а до Java 8 — O(n).

Худшее время get(key) для ключа, которого нет, и для ключа, который есть?

В обоих случаях всё определяет длина корзины: O(1) в среднем, O(log n) при дереве в Java 8+, O(n) до Java 8 или при несравнимых ключах с одинаковым хэшем.

Может ли HashMap выродиться в список при разных hashCode()?

Да: индекс — `(n – 1) & hash`, то есть только младшие биты. Разные хэши с одинаковыми младшими битами (1, 17, 33 при 16 корзинах) попадут в одну корзину. Подмешивание `h >>> 16` уменьшает, но не исключает такие случаи.

Будет ли работать HashMap, если у всех ключей одинаковый hashCode()?

Да, корректно, но медленно: все ключи в одной корзине — O(log n) при дереве, O(n) для несравнимых ключей.

Как ключи без Comparable сравниваются в корзине-дереве?

По `hash`, затем `compareTo` (если ключи `Comparable` одного класса), иначе `tieBreakOrder` — по имени класса и `System.identityHashCode`.

Когда и как HashMap расширяется? Смысл initialCapacity и loadFactor.

Когда `size > capacity × loadFactor` — таблица удваивается, узлы делятся между индексами `i` и `i + oldCapacity`. `initialCapacity` — начальное число корзин, `loadFactor` — допустимая заполненность (0.75): меньше — меньше коллизий, но больше памяти.

Сколько объектов создаётся при добавлении нового элемента в HashMap?

Один `Node` (при первом `put` — ещё массив корзин; при расширении — новый массив).

Сколько «шагов» делает get(key) для существующего ключа?

Вычислить хэш, найти индекс корзины, сравнить с первым узлом (`hash` и `equals`) и вернуть значение. Если ключ не первый — ещё проход по цепочке или дереву. `null`-ключ всегда в корзине 0.

Когда можно потерять элемент в HashMap?

Если изменить поле ключа, участвующее в `hashCode`, после вставки: поиск идёт в другую корзину по новому хэшу.

Почему byte[] не подходит как ключ HashMap?

У массивов `equals` и `hashCode` от `Object` — по ссылке. Массив с тем же содержимым ключ не найдёт. Оборачивать в `ByteBuffer`, `List` или `record` с `Arrays.equals`.

Максимальное число значений hashCode()?

2^{32} — диапазон `int`.

Метод цепочек или открытая адресация — что в HashMap и почему?

Цепочки: простое удаление, плавная деградация при высоком заполнении, возможность превращать цепочки в деревья. Открытую адресацию использует `IdentityHashMap`.

HashMap vs Hashtable? Зачем HashMap, если есть Hashtable?

`Hashtable` синхронизирует каждый метод и не допускает `null`; `HashMap` быстрее, допускает один `null`-ключ и `null`-значения, итераторы fail-fast. `Hashtable` устарел, в многопоточном коде — `ConcurrentHashMap`.

HashMap vs IdentityHashMap? Для чего IdentityHashMap?

`IdentityHashMap` сравнивает ключи по `==` и `identityHashCode`, внутри — открытая адресация. Нужна, когда важны конкретные объекты, а не равенство: обход графа объектов, сериализация, глубокое копирование.

HashMap vs WeakHashMap? Для чего WeakHashMap?

Ключи `WeakHashMap` — слабые ссылки: когда на ключ не остаётся сильных ссылок, запись исчезает после GC. Для данных, прикреплённых к чужим объектам, без утечек памяти.

Почему нет SoftHashMap и PhantomHashMap?

SoftHashMap есть в сторонних библиотеках, но memory-sensitive кэши лучше делать в Caffeine. PhantomHashMap бессмысленна: `PhantomReference.get()` всегда возвращает `null`.

LinkedHashMap — что от LinkedList, что от HashMap?

От `HashMap` — хэш-таблица и O(1), от `LinkedList` — двусвязный список через все записи для порядка обхода: вставки или доступа (`accessOrder = true`). Повторный `put` существующего ключа порядок вставки не меняет.

В чём «сортированность» SortedMap, кроме toString()?

Обход по возрастанию ключей, `firstKey()` / `lastKey()`, диапазоны `headMap` / `tailMap` / `subMap`; у `NavigableMap` — `floorKey`, `ceilingKey` и обратный порядок.

Как перебрать ключи, значения и пары Map?

`keySet()`, `values()`, `entrySet()` (или `forEach((k, v) -> ...)`). Для пар — `entrySet()`, а не `keySet()` + `get`.

Set и сортировка — вопросы

TreeSet vs HashSet?

`HashSet` — на `HashMap`: O(1), порядок не определён, равенство по `equals` / `hashCode`. `TreeSet` — на `TreeMap`: O(log n), отсортирован, равенство по `compareTo`, `null` нельзя.

Что будет, если добавлять элементы в TreeSet по возрастанию?

Ничего плохого: красно-чёрное дерево само балансируется, операции остаются O(log n).

LinkedHashSet vs HashSet?

`LinkedHashSet` построен на `LinkedHashMap` и обходит элементы в порядке вставки; повторное добавление порядок не меняет.

Зачем EnumSet, если есть HashSet и TreeSet?

Битовый вектор по `ordinal()`: в разы компактнее, операции O(1) битовыми масками, массовые операции (`containsAll`, `retainAll`) — одна операция над `long`, обход в порядке объявления.

Comparable vs Comparator?

`Comparable.compareTo` — естественный порядок, заданный самим классом. `Comparator.compare` — внешний порядок; их может быть много, и можно сортировать чужие классы.

Почему в compareTo нельзя писать a - b?

Переполнение `int`: при больших по модулю значениях знак разности неверный. Используем `Integer.compare(a, b)` или `Comparator.comparingInt`.

Что значит «компаратор согласован с equals»?

`compare(a, b) == 0` тогда и только тогда, когда `a.equals(b)`. Иначе `TreeSet` и `HashSet` по-разному считают дубликаты (пример — `BigDecimal` 2.0 и 2.00).

Неизменяемые и потокобезопасные — вопросы

Как получить коллекцию только для чтения?

`List.of` / `List.copyOf` — неизменяемая копия. `Collections.unmodifiableList` — представление только для чтения: изменения исходного списка через него видны.

Чем List.of отличается от Arrays.asList?

`List.of` полностью неизменяемый и запрещает `null`. `Arrays.asList` — фиксированный размер поверх массива: `set` работает и пишет в массив, `add` / `remove` бросают UOE, `null` можно.

Как получить синхронизированную коллекцию?

`Collections.synchronizedList / Map / Set` — каждый метод синхронизирован, но перебор и составные операции нужно синхронизировать вручную. Лучше concurrent-коллекции: `ConcurrentHashMap`, `CopyOnWriteArrayList`.

Приведите пример UnsupportedOperationException.

`Collections.emptyList().add(0)`, `List.of(1).add(2)`, `Arrays.asList(1, 2).remove(0)`, `iterator.remove()` y `CopyOnWriteArrayList`.

Задачи с кодом

Однопоточная программа, которая бросает ConcurrentModificationException

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
for (Integer x : list) {
    list.remove(x);      // for-each использует итератор → CME на следующем next()
}
```

Симметрическая разность двух коллекций через addAll, removeAll, retainAll

```
static <T> Set<T> symmetricDifference(Collection<T> a, Collection<T> b) {
    Set<T> result = new HashSet<>(a);   // A ∪ B
    result.addAll(b);
    Set<T> common = new HashSet<>(a);   // A ∩ B
    common.retainAll(b);
    result.removeAll(common);           // (A ∪ B) − (A ∩ B)
    return result;                      // [1, 2, 3] и [2, 3, 4] → [1, 4]
}
```

Через `Set`, а не `ArrayList`: иначе дубликаты внутри одной коллекции дадут неверный результат.

LRU-кэш на LinkedHashMap

```
Map<K, V> cache = new LinkedHashMap<>(16, 0.75f, true)
{
    protected boolean removeEldestEntry(Map.Entry<K, V>
eldest) {
        return size() > MAX;
    }
};
```

Скопировать любую коллекцию в массив одной строкой

```
Object[] arr = collection.toArray();
String[] typed = collection.toArray(String[]::new);
// Java 11
```

List без первых и последних трёх элементов одним вызовом

```
List<Integer> middle = list.subList(3, list.size() -
3);   // представление, а не копия
```

HashSet → ArrayList и ArrayList → HashSet одной строкой

```
List<Integer> list = new ArrayList<>(set);
Set<Integer> set = new HashSet<>(list);
```

HashSet из ключей HashMap

```
Set<K> keys = new HashSet<>(map.keySet());   // копия;
map.keySet() — живое представление
```

HashMap из HashSet<Map.Entry<K, V>>

```
Map<K, V> map = entries.stream()
        .collect(Collectors.toMap(Map.Entry::getKey,
Map.Entry::getValue));

Map<K, V> map2 = new HashMap<>(entries.size() * 4 / 3 +
1);   // или циклом
for (Map.Entry<K, V> e : entries) map2.put(e.getKey(),
e.getValue());
```

Топ-K самых больших чисел

```
PriorityQueue<Integer> heap = new PriorityQueue<>();
// min-heap размера k
for (int x : nums) {
    heap.offer(x);
    if (heap.size() > k) heap.poll();
// выбрасываем самый маленький
}
// O(n log k)
```